# Day 49 -- Line-by-Line Syntax Explanation

day49_alexnet_style_cnn.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 7, Day 9 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula, a purple MATH box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green SYSTEM DIAGRAM shows how that math and code fit together as a working system.

***MATH box legend:*** *× = matrix multiply / elementwise times  W, b = weights/bias  m = batch size  lr = learning rate  d(name) = gradient of loss w.r.t. that tensor*

## Segment 0 -- Imports and Shared Helpers (reused verbatim from Days 39-48)

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
np.random.seed(42)
def relu(z): return np.maximum(0, z)
def drelu(z): return (z > 0).astype(z.dtype)
def softmax(z): ...
def one_hot(y, n_classes): ...
def cross_entropy_loss(probs, y_onehot): ...
def he_scale(n_in): return np.sqrt(2.0 / n_in)
def conv2d_forward_mc(X, W, b): ...
def conv2d_backward_mc(dZ, X, W): ...
def maxpool_forward(A, size=2, stride=2): ...
def maxpool_backward(dOut, A, idx_cache, size=2, stride=2): ...
def iterate_minibatches(X, y_onehot, batch_size, rng, shuffle=True): ...
def train_minibatch(net, X, y, epochs, batch_size, n_classes=4, seed=0): ...
def make_junction_dataset_rgb(...): ...
class TwoBlockConvNetRGB: ...
```

None of this is new today -- every function here is a character-for-character copy of earlier days' own code: the conv/pool/loss helpers from Days 39-47, and iterate_minibatches / train_minibatch from Day 45, unchanged. TwoBlockConvNetRGB plays the baseline role today, trained via train_minibatch (it has no dropout, so it needs no training-mode flag). make_junction_dataset_rgb's size argument is simply passed 40 today instead of the usual 30 -- the function body itself never changes.

## Segment 1 -- Dropout, a Standalone Layer (NEW today)

**MATH**

```
training=True: mask = (rand(*A.shape) < keep_prob) / keep_prob
out = A * mask (E[out] = A, unbiased)
training=False: out = A, mask = None (no rescaling at all)
backward: dA = dOut * mask (mask=None -> unchanged)
```

```
def dropout_forward(A, keep_prob, training, rng):
    if not training or keep_prob >= 1.0:
        return A, None
    mask = (rng.rand(*A.shape) < keep_prob) / keep_prob
    return A * mask, mask

def dropout_backward(dOut, mask):
    if mask is None:
        return dOut
    return dOut * mask
```

The early-return `if not training or keep_prob >= 1.0: return A, None` makes both no-op conditions (eval mode, or keep_prob=1.0 during training) provably identical to skipping dropout entirely -- not just intended to behave that way. `rng.rand(*A.shape)` draws one uniform(0,1) value per unit; comparing `< keep_prob` turns that into a boolean survive/drop mask with the right probability, and dividing by keep_prob in the SAME expression is what makes this inverted (rather than plain) dropout. dropout_backward's `mask=None` branch means the exact same function works whether or not dropout was active on a given forward call -- callers never need their own if/else for it.

> **SYSTEM DIAGRAM -- one dropout_forward call -- training=True (random mask, rescaled) vs. training=False (identity).**

## Segment 2 -- AlexNetStyleConvNet (NEW today)

```
class AlexNetStyleConvNet:
    def __init__(self, f1=4, f2=8, f3=16, k=3, img_size=40, hidden1=32,
                 hidden2=16, n_classes=4, lr=0.05, keep_prob=0.5, seed=42):
        ...
        out1 = img_size - k + 1; pool1 = out1 // 2
        out2 = pool1 - k + 1;    pool2 = out2 // 2
        out3 = pool2 - k + 1;    pool3 = out3 // 2
        self.flat_dim = f3 * pool3 * pool3
```

The `out // 2` pattern (integer floor division) is reused unchanged from TwoBlockConvNetRGB and LeNet5Style -- it correctly predicts maxpool_forward's real output size for size=2, stride=2, for both even and odd input dimensions, which is why the constructor can compute self.flat_dim ahead of time instead of running a dummy forward pass to discover it. A THIRD block (out3/pool3) is the only structural addition versus the 2-block baseline's __init__.

```
def forward(self, X, training=False, rng=None):
    rng = rng if rng is not None else np.random
    ...
    self.A1 = relu(self.Z1)
    self.A1d, self.mask1 = dropout_forward(self.A1, self.keep_prob, training, rng)
    self.Z2 = self.A1d @ self.W2 + self.b2
    self.A2 = relu(self.Z2)
    self.A2d, self.mask2 = dropout_forward(self.A2, self.keep_prob, training, rng)
    self.Z3 = self.A2d @ self.W3 + self.b3
    self.probs = softmax(self.Z3)
    return self.probs
```

`forward` now takes `training` and `rng` parameters that no earlier network's forward() needed -- this is the same explicit training-flag convention Day 38's RegNet used, generalized into its own layer. `rng if rng is not None else np.random` lets forward() be called for a quick one-off check (no rng given, falls back to the global np.random state) or as part of a reproducible training run (an explicit np.random.RandomState passed in) without two separate code paths. Both dropout calls cache their own mask (self.mask1, self.mask2) on self, exactly like maxpool caches its own index masks -- backward() needs them and they must not be recomputed (recomputing would draw a DIFFERENT random mask than forward actually used).

```
def backward(self, y_onehot):
    ...
    dA2d = dZ3 @ self.W3.T
    dA2 = dropout_backward(dA2d, self.mask2)
    dZ2 = dA2 * drelu(self.Z2)
    ...
    dA1d = dZ2 @ self.W2.T
    dA1 = dropout_backward(dA1d, self.mask1)
    dZ1 = dA1 * drelu(self.Z1)
```

Each dropout_backward call sits between the matmul gradient (dA2d, dA1d) and the ReLU gradient (drelu) it feeds -- mirroring forward's own order (ReLU, then dropout) exactly reversed, the same chain-rule bookkeeping every other backward() in this series follows.

```
def train(self, X, y, epochs, batch_size, n_classes=4, seed=0):
    y_oh = one_hot(y, n_classes)
    rng = np.random.RandomState(seed)
```

```
    step_losses = []
    for epoch in range(epochs):
        for Xb, yb, idx in iterate_minibatches(X, y_oh, batch_size, rng):
            probs = self.forward(Xb, training=True, rng=rng)
            step_losses.append(cross_entropy_loss(probs, yb))
            self.backward(yb)
    return step_losses
```

The SAME `rng` object drives both the mini-batch shuffling (inside iterate_minibatches) and every dropout mask drawn during training -- one RandomState, advancing sequentially, is what makes a full training run reproducible end to end from a single seed. `training=True` is passed explicitly on every training-step forward call; `accuracy()` (not shown) calls forward with training=False, so evaluation never applies dropout.

**SYSTEM DIAGRAM -- AlexNetStyleConvNet.forward, training=True path -- 3 conv+pool blocks feeding 2 dropout-regularized FC layers into softmax.**

## Segment 3 -- Dropout Unit Check

```
n_trials = 2000
sum_out = np.zeros_like(A)
for t in range(n_trials):
    out, mask = dropout_forward(A, keep_prob, training=True, rng=rng)
    sum_out += out
    zero_fracs[t] = np.mean(mask == 0)
mean_out = sum_out / n_trials
grand_rel_err = abs(mean_out.sum() - A.sum()) / abs(A.sum())
```

`sum_out += out` accumulates across all 2,000 trials rather than storing every trial's full output array -- the running-sum approach uses O(A.size) memory instead of O(n_trials * A.size). `grand_rel_err` sums BOTH `mean_out` and `A` down to single scalars before comparing -- pooling every one of the 100,000 units' 2,000-sample averages into one number, which is what makes this statistic converge so much tighter than any individual unit's own average (reported separately as `per_unit_rel_err`, and expected to stay noisy).

## Segment 4 -- Gradient Check (three-deep pipeline, dropout off)

```
net = AlexNetStyleConvNet(img_size=24, n_classes=3, lr=0.01,
                          keep_prob=1.0, seed=1)
...
net.forward(X, training=False)
```

`keep_prob=1.0` and `training=False` together guarantee dropout_forward's early-return path runs on every call in this check -- mask is always None, so repeated forward() calls at a perturbed parameter (+eps, then -eps) are deterministic. Everything from here mirrors Day 47's own gradient-check structure: hand-derive the analytic gradient by walking the same chain forward() built, then compare it against a central-difference numeric estimate at a handful of random indices per tensor -- now across FOUR tensors (Wc1, Wc3, W1, W3) spanning all three conv blocks plus both FC layers.

## Segment 5 -- Training Comparison + Plot

```
alexnet = AlexNetStyleConvNet(img_size=img_size, lr=0.05, keep_prob=0.5, seed=42)
losses_alexnet = alexnet.train(X_train, y_train, epochs=40, batch_size=50, seed=42)
...
base = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=img_size, hidden=8, lr=0.05, seed=42)
losses_base = train_minibatch(base, X_train, y_train, epochs=40, batch_size=50, seed=42)
```

AlexNetStyleConvNet calls its OWN `.train()` method (it needs the training=True flag for dropout), while the baseline is trained by handing it to Day 45's generic `train_minibatch()` function instead -- the baseline has no dropout, so it needs no training-mode flag, and reusing the existing generic function instead of writing a near-duplicate `.train()` method for it keeps the comparison honest: both networks see the identical batch_size, epoch count, and shuffling mechanics.

**SYSTEM DIAGRAM -- the mini-batch step loop both networks share: iterate_minibatches yields shuffled batches; each calls forward()->loss->backward() once per batch.**

## Segment 6 -- Dropout Ablation + Plot

```
net_dropout = AlexNetStyleConvNet(img_size=img_size, lr=0.05, keep_prob=0.5, seed=42)
losses_dropout = net_dropout.train(X_train, y_train, epochs=40, batch_size=50, seed=42)
...
net_nodrop = AlexNetStyleConvNet(img_size=img_size, lr=0.05, keep_prob=1.0, seed=42)
losses_nodrop = net_nodrop.train(X_train, y_train, epochs=40, batch_size=50, seed=42)
```

Every argument is identical between the two networks except `keep_prob` -- same seed (so both start from the same initial weights), same data, same epoch/batch_size schedule. This is what makes the resulting train_acc/test_acc difference attributable to dropout alone, rather than to some other, accidental difference between the two runs.

## Segment 7 -- What This Maps To in PyTorch, and main()

```
def note_on_pytorch():
    print(
        "class AlexNetStyle(nn.Module):\n"
        "    def __init__(self, n_classes=4, p_drop=0.5):\n"
        ...
    )

def main():
    demo_dropout_unit_check()
    demo_gradient_check()
    data = demo_train_and_compare()
    demo_dropout_ablation(data)
    note_on_pytorch()
```

`note_on_pytorch` is a pure documentation function, same pattern every prior day's own version has used -- it prints a PyTorch equivalent as a string rather than importing torch. `main()` runs correctness checks FIRST (dropout unit check, then the full gradient check), exactly the same ordering principle every earlier day has followed: verify the new code is right before trusting any number it produces. `demo_train_and_compare()` returns its train/test arrays in a dict (`data`), which `demo_dropout_ablation(data)` reuses directly -- the exact same 40x40 dataset is never regenerated a second time, so both comparisons in this lesson run on byte-identical data.

## Quick Syntax Glossary -- New or Reinforced Constructs

**`(rng.rand(*A.shape) < keep_prob) / keep_prob` (one expression, two jobs)**

The comparison `< keep_prob` produces a boolean survive/drop mask; dividing that SAME array by keep_prob in the same line is what turns plain dropout into INVERTED dropout. Doing both in one expression (rather than a separate rescaling step later) makes it structurally impossible to apply the mask without also applying its rescaling.

**`rng = rng if rng is not None else np.random` (optional-dependency default)**

New today: a function parameter that defaults to a MODULE (np.random) rather than None or a literal, when the caller doesn't need reproducibility. Reused code (a quick sanity call) can omit `rng` entirely; a real training run always passes an explicit RandomState.

**`self.mask1`, `self.mask2` cached on `self` (mirrors `self.masks1` from maxpool)**

Dropout's random mask, like maxpool's argmax indices, MUST be reused by backward() rather than redrawn -- redrawing would silently compute gradients for a different forward pass than the one that actually ran. This is the same cache-on-self pattern every stateful layer in this series already follows.

**`training=True` / `training=False` as an explicit function argument**

Generalizes Day 38's inline `training=` flag (there, hardcoded into one class) into a convention any layer can follow: dropout_forward takes it directly rather than reading a class-level mode flag, so the SAME function serves both a training step and an evaluation call correctly.

**`mean_out.sum()` vs. per-unit `mean_out` (pooling before comparing)**

Collapsing a whole array down to one scalar with `.sum()` before computing a relative error is a deliberate choice to test an AGGREGATE statistical claim (the pooled mean matches A) rather than a per-element one -- appropriate here specifically because dropout's per-unit noise is expected to be large at any practical sample count.

---

*End of Day 49 syntax reference. For the concepts these lines implement, see the main Day 49 study guide PDF.*